The Logical Foundations of the Cathedral: Type Theory, Trust, and the Metatheory of a Machine-Verified Reduction

Jason Robert Gochanour

April 19, 2026

Abstract

We analyze the logical foundations of the Cathedral, a Lean 4 formalization establishing RH $\iff$ $d_N^2 \to 0$. The proof inhabits the Calculus of Inductive Constructions (CIC) extended with three kernel axioms (`propext`, `Classical.choice`, `Quot.sound`) and two mathematical axioms (`bd_mellin_at_zero`, `rh_implies_l2_convergence`). We establish the precise logical strength of the formalization: it is provable in ZFC plus the consistency of the kernel axioms (which is equiconsistent with ZFC). We analyze the trust chain from silicon to theorem, examine whether the proof can be made constructive (partially: the converse but not the forward direction), and compare the foundational requirements with formalizations in Coq, Isabelle/HOL, and Mizar. We argue that the Cathedral's axiomatic architecture provides a template for *foundationally transparent* formalization of open problems, where the logical overhead is separated from the mathematical content with surgical precision.

Contents

1 Introduction: What Lies Beneath the Proof

Every machine-verified proof rests on a stack of trust:

Layer	What Must Be Trusted
5	Mathematical axioms (2 in the Cathedral)
4	Mathlib library (∼1.4M LOC)
3	Lean kernel axioms (3: `propext`, `choice`, `Quot.sound`)
2	Lean type-checker (∼5,000 LOC of C++ kernel)
1	Compiler, OS, hardware

The Cathedral lives at Layer 5. Its correctness depends on every layer below. This paper examines Layers 2–4 in detail, characterizes the logical strength of the entire stack, and asks: what exactly are we trusting when we trust the Cathedral?

2 The Type Theory: CIC with Extensions

2.1 The Core Calculus

Lean 4 is based on a variant of the Calculus of Inductive Constructions (CIC), a dependent type theory with:

- **Dependent function types** (Π): $\forall(x : A), B(x)$.
- **Inductive types**: $\mathbb{N}$, `List`, `Prop`, etc.
- **A universe hierarchy**: $\texttt{Prop} : \texttt{Type}\,0 : \texttt{Type}\,1 : \cdots$
- **Proof irrelevance**: All terms of type $P : \texttt{Prop}$ are definitionally equal.
- **Large elimination**: Pattern matching on inductive types can produce types, not just terms.

The consistency of CIC (without classical extensions) follows from its normalization, which was proved by Werner (1997) for the system underlying Coq. Lean 4's variant differs in technical details (quotient types, mutual inductives) but is believed equiconsistent.

2.2 The Three Kernel Axioms

Lean 4 adds three axioms to the core CIC:

Definition 2.1 (`propext`)**.** Propositional extensionality: $(P \iff Q) \to P = Q$ for $P, Q : \texttt{Prop}$.

This is essential for working with sets (defined as predicates). Without it, two logically equivalent propositions are not considered equal, making set-theoretic reasoning extremely verbose.

Definition 2.2 (`Classical.choice`)**.** The axiom of choice (in a type-theoretic formulation): for any nonempty type A, there exists an element $a : A$. Formally: $\forall (A : \texttt{Sort u}),\ \texttt{Nonempty}\, A \to A$.

This axiom implies the law of excluded middle ($P \vee \neg P$ for all $P : \texttt{Prop}$) and enables classical reasoning. It also enables the `noncomputable` keyword: definitions that use `choice` cannot be executed but can be reasoned about.

Definition 2.3 (`Quot.sound`)**.** Quotient soundness: if $a \sim b$ then $[a] = [b]$ in the quotient type $A/\sim$. This is built into Lean's type theory as a primitive, not derivable from other axioms.

Proposition 2.4 (Consistency)**.** *The system* CIC+*`propext`*+*`choice`*+*`Quot.sound`* *is equiconsistent with* ZFC*. That is: if* ZFC *is consistent, then the extended CIC is consistent, and vice versa.*

This is a folklore result in type theory. The forward direction follows from the set-theoretic model of CIC; the reverse from the interpretation of ZFC in CIC with large cardinals (which are not needed here).

3 The Five Axioms of the Cathedral

The crown theorem `nyman_beurling_equivalence` reports exactly 5 axioms via `#print axioms`:

#	Axiom	Origin
1	`propext`	Lean kernel
2	`Classical.choice`	Lean kernel
3	`Quot.sound`	Lean kernel
4	`bd_mellin_at_zero`	Cathedral
5	`rh_implies_l2_convergence`	Cathedral

Axioms 1–3 are shared by every Lean 4 proof that uses Mathlib. They are part of the logical infrastructure, not the mathematical content. The foundationally interesting axioms are 4 and 5.

3.1 Axiom 4: `bd_mellin_at_zero`

This axiom asserts the analytic continuation of the Mellin transform identity for the Báez-Duarte basis function:

$$\int_0^1 \{1/(kx)\}\, x^{s-1}\, dx = \frac{k^{-s}}{s(s-1)}\big(s\zeta(s+1)/k - (s-1)\zeta(s)/k + 1\big) \quad (\operatorname{Re} s > 0, s \neq 1).$$

This is proved for $\operatorname{Re} s > 1$ by direct computation (telescoping sums over the floor function). The extension to $\operatorname{Re} s > 0$ requires analytic continuation: both sides are analytic on $\{s \in \mathbb{C} : \operatorname{Re} s > 0, s \neq 1\}$ and agree on $\{s : \operatorname{Re} s > 1\}$, so they agree everywhere by the identity theorem.

Remark 3.1 (Logical strength)**.** This axiom is provable in ZFC (and indeed in much weaker systems). It requires no set-theoretic strength beyond the existence of the Riemann zeta function and the identity theorem for analytic functions, both of which are in Mathlib. The axiom is a *formalization gap*, not a foundational one: Mathlib simply lacks the specific lemma connecting Mellin integrals to analytic continuation.

3.2 Axiom 5: `rh_implies_l2_convergence`

This axiom asserts the forward direction of the Nyman–Beurling criterion:

$$\text{RH} \implies \forall\, \varepsilon > 0,\ \exists\, N_0,\ \forall\, N \geq N_0,\ \exists\, v : \quad d_N^2(v) < \varepsilon.$$

This is the Báez-Duarte theorem. Its proof requires:

1. The Mertens function bound $|M(x)| \leq C\sqrt{x}\log^2 x$ under RH.
2. Abel summation to convert the Mertens bound into a Dirichlet polynomial bound.
3. The Montgomery–Vaughan mean value theorem for controlling L^2 norms of Dirichlet polynomials on the critical line.

Remark 3.2 (Logical strength)**.** This axiom is also provable in ZFC. The deepest ingredient (Montgomery–Vaughan) uses only standard analytic tools: the large sieve inequality, Perron's formula, and the functional equation of $\zeta(s)$, all of which are elementary from a foundational perspective.

3.3 The Cathedral Is Provable in ZFC

Theorem 3.3 (Foundational Location)**.** *The crown theorem* RH $\iff$ $d_N^2 \to 0$*, as stated in the Cathedral, is provable in* ZFC *(indeed in much weaker systems). The 2 mathematical axioms are formalization gaps, not foundational gaps. The 3 kernel axioms are equiconsistent with* ZFC*.*

This means the Cathedral does not require any large cardinal axioms, forcing arguments, or other foundational exotica. The theorem is ordinary mathematics, verified by a machine built on ordinary foundations.

4 Constructivity Analysis

Can the Cathedral be made constructive? That is: can the proof be carried out without `Classical.choice` (and hence without the law of excluded middle)?

4.1 The Converse: Partially Constructive

The converse direction ($d_N^2 \to 0 \implies$ RH) is proved via the Rank-1 Mellin Miracle, which proceeds by contrapositive:

If ¬RH, then $\exists\, \rho$ with $\zeta(\rho) = 0$ and $\operatorname{Re}\rho \neq 1/2$, so $d_N^2 \geq \delta > 0$ for all N.

The contrapositive uses classical logic (specifically, the equivalence $(\neg Q \implies \neg P) \iff (P \implies Q)$). In constructive logic, contrapositive reasoning is valid only for ¬¬-stable propositions.

However, the *bound* $d_N^2 \geq \delta > 0$ *is* constructive: given ρ, the bound is computed explicitly. The non-constructive step is the existential quantifier over ρ in ¬RH.

Proposition 4.1. *The converse* $d_N^2 \to 0 \implies$ RH *can be proved in constructive type theory (CIC without* `choice`*) if one formulates* RH *as* $\forall\, s,\ \zeta(s) = 0 \implies \operatorname{Re} s = 1/2$ *(a universally quantified statement) rather than as a* ¬∃ *statement.*

4.2 The Forward: Inherently Classical

The forward direction (RH $\implies$ $d_N^2 \to 0$) uses the Mertens function bound, which is proved using contour integration and the residue theorem. These tools are inherently classical (they rely on the law of excluded middle for existence of convergent subsequences, compactness arguments, etc.).

Proposition 4.2. *The forward direction* RH $\implies$ $d_N^2 \to 0$ *is not known to be provable in constructive type theory. The deepest classical ingredient is the Montgomery–Vaughan mean value theorem, which uses the large sieve inequality and Perron's formula.*

4.3 Use of Choice in Mathlib

Even if the Cathedral's mathematical content could be made constructive, Mathlib's infrastructure uses `Classical.choice` pervasively:

- `MeasureTheory.integral` is defined using `Classical.choice` (to select representatives).
- `Matrix.nonsing_inv` uses `Classical.choice` (to decide invertibility).
- `Real.log` and `Real.sqrt` use `Decidable` instances derived from `Classical.choice`.

Removing `choice` would require replacing virtually every Mathlib API used in the Cathedral.

5 The Trust Chain

5.1 The de Bruijn Criterion

The Cathedral satisfies the *de Bruijn criterion*: its correctness depends only on a small, auditable kernel. Lean 4's type-checker kernel is approximately 5,000 lines of C++ code. An independent implementation (Lean's *lean4checker* tool) can verify the same proofs, providing redundancy.

Component	**Size (LOC)**
Lean kernel (type-checker)	∼5,000
Lean elaborator + tactics	∼100,000
Mathlib library	∼1,400,000
Cathedral	∼20,000

Only the kernel (5,000 LOC) is in the trusted base. The elaborator, tactics, and Mathlib could be buggy—if so, the kernel would reject the proof. The Cathedral could be wrong—if so, `lake build` would fail.

5.2 Potential Failure Points

For the Cathedral to be incorrect, one of the following would need to be true:

1. **The Lean kernel has a soundness bug.** Possible but extremely unlikely; the kernel is small, well-tested, and has been independently re-implemented.
2. **The hardware produced a wrong result.** Possible (cosmic rays, etc.) but mitigated by deterministic rebuilds on different machines.

3. **The mathematical axioms are false.** Both axioms are provable in ZFC by standard methods. If either is false, there is a fundamental error in classical analytic number theory.

4. **The statement was formalized incorrectly.** This is the most realistic risk: the Lean type `RiemannHypothesis` might not faithfully represent the classical RH. Mathlib's `riemannZeta` is carefully constructed from Dirichlet series and analytic continuation, matching the standard definition.

Remark 5.1 (The Specification Problem)**.** Risk (4) cannot be fully eliminated by any formal system. The bridge from informal mathematics to formal types is inherently informal. However, the Cathedral's formalization of RH uses Mathlib's `riemannZeta`, which has been audited by the Lean community and matches the standard definition via the Dirichlet series $\sum n^{-s}$ plus analytic continuation.

6 Comparison with Other Proof Assistants

How would the Cathedral differ if formalized in a different system?

	Lean 4	**Coq**	**Isabelle/HOL**	**Mizar**
Type theory	CIC + ext	CIC	HOL	Set theory
Classical logic	Axiom	Optional	Built-in	Built-in
Zeta function	Mathlib	Absent	Absent	Absent
Matrix algebra	Mathlib	MathComp	HOL-Analysis	MML
Proof style	Tactic	Tactic	Isar	Declarative
Trust base	5K LOC	8K LOC	10K LOC	40K LOC

6.1 Coq

A Coq formalization would use the same core type theory (CIC) but would need to import classical axioms explicitly (via `Classical` or `Require Import ClassicalEpsilon`). The main barrier is that Coq lacks Mathlib's analysis library: there is no `riemannZeta` in Coq. The MathComp library provides excellent algebra but limited analysis.

A Coq formalization would have one advantage: the option to identify which parts of the proof are constructive (using Coq's extraction mechanism). The converse direction could potentially be extracted to a program.

6.2 Isabelle/HOL

Isabelle's HOL (Higher-Order Logic) is a simple type theory, not a dependent type theory. It is classical by default, with a small kernel and strong proof automation (`auto`, `sledgehammer`). HOL-Analysis provides real analysis but lacks the complex analysis needed for the zeta function.

An Isabelle formalization would be more automated (Sledgehammer would close many intermediate goals) but would require significant library development for the zeta function.

6.3 Mizar

Mizar uses a set-theoretic foundation (Tarski–Grothendieck set theory), which is closest to how mathematicians actually think. The Mizar Mathematical Library (MML) has some number theory but lacks modern analytic tools. Mizar's declarative proof style would produce a formalization that reads more like traditional mathematics but is harder to develop interactively.

6.4 Assessment

Lean 4 is currently the only system where the Cathedral is feasible without massive library development, because Mathlib is the only library that provides: (a) the Riemann zeta function, (b) L^2 integration, (c) matrix algebra, and (d) measure theory, all in a single coherent framework.

7 Universe Levels and Large Eliminations

The Cathedral uses Lean 4's universe polymorphism minimally. The key universe considerations:

- All mathematical content lives in `Type 0` (the universe of small types). The Gram matrix indices are `Fin N`, which is in `Type 0`.
- `Prop` is used for all logical statements. Proof irrelevance ensures that the specific proof term is irrelevant; only the type (the statement) matters.
- No large eliminations are used in the critical proof path. The proof does not construct types by pattern-matching on proof terms.
- Universe polymorphism appears only in Mathlib's generic infrastructure (e.g., `Module R M` is universe-polymorphic).

The foundational simplicity here is deliberate: the Cathedral avoids universe-level complications, keeping the proof in the fragment of CIC that is best understood and most widely trusted.

8 The Axiom-Theorem Gradient

The Cathedral's axiomatic architecture suggests a foundational concept: the *axiom-theorem gradient*. In a traditional formalization, every statement is either an axiom (accepted without proof) or a theorem (derived from axioms). The Cathedral introduces a finer gradation:

1. **Kernel axioms**: Trusted by design; part of the logic. (3 in the Cathedral.)
2. **Active mathematical axioms**: Believed true, not yet formalized. On the critical path. (2 in the Cathedral.)
3. **Off-path axioms**: Formalization gaps that do not affect the crown theorem. (38 in the Cathedral.)
4. **Theorems**: Machine-verified from axioms and Mathlib. (644 in the Cathedral.)
5. **Archived axioms**: Previously active, now proved and excised. (16 eliminated during the project.)

This gradient makes the *progress* of formalization visible. Each axiom elimination moves a statement from category 2 or 3 to category 4, with category 5 providing a historical record. The gradient is not a feature of the type theory—it is a *project management* concept layered on top of the foundations.

Remark 8.1. The axiom-theorem gradient is analogous to the debt-equity spectrum in finance: axioms are "debt" (promises to prove later) and theorems are "equity" (fully owned mathematical knowledge). The Cathedral's reduction from 56 axioms to 40 (with 2 on the critical path) is a deleveraging of the proof's balance sheet.

9 What Would Change If RH Were Proved?

If the Riemann Hypothesis were proved (by any means), what would happen to the Cathedral?

1. **Axiom 5 becomes a theorem.** The forward direction RH $\implies$ $d_N^2 \to 0$ would be provable, and the axiom would be eliminated. The crown theorem would depend on 1 mathematical axiom.

2. **Axiom 4 might also fall.** The analytic continuation axiom is independent of RH and could be formalized separately. If both axioms were eliminated, the crown theorem would depend on zero mathematical axioms—only the 3 kernel axioms.

3. **The biconditional would become a theorem of Lean 4 + Mathlib.** It would join the ranks of the Four-Color Theorem, the Kepler Conjecture, and the Odd Order Theorem as major results formalized in proof assistants.

Conversely, if RH were *disproved* (a zero found off the critical line), the Cathedral's biconditional would imply $d_N^2 \not\to 0$: the Báez-Duarte distance would be bounded away from zero. This would be a constructive consequence of the Cathedral plus the counterexample.

10 Conclusion: Foundational Transparency

The Cathedral demonstrates that large-scale formalization of frontier mathematics can be *foundationally transparent*: every assumption is named, classified, and tracked, from the kernel axioms at the bottom to the mathematical axioms at the top. The trust chain is auditable. The logical strength is characterized. The constructive content is identified.

This transparency is not automatic. It required deliberate architectural decisions: the axiom registry, the tiered classification, the `#print axioms` discipline, the ghost axiom audits. Without these, the formalization would still type-check but its foundational status would be opaque.

We propose *foundational transparency* as a design principle for future formalizations of open problems. The goal is not merely to produce a proof that type-checks, but to produce a proof whose *foundational commitments* are visible, minimal, and precisely documented.

> *A proof is not just a certificate of truth. It is a map of trust, showing exactly where certainty ends and faith begins.*

Source code: `https://github.com/jrgochan/prime`

References

[1] L. de Moura et al., *The Lean 4 Theorem Prover and Programming Language*, CADE-28, 2021.

[2] B. Werner, *Sets in Types, Types in Sets*, Proc. TACS 1997.

[3] M. Carneiro, *The Type Theory of Lean*, MSc thesis, Carnegie Mellon University, 2019.

[4] N.G. de Bruijn, *The Mathematical Language AUTOMATH*, Symposium on Automatic Demonstration, Springer LNCS 125 (1970).

[5] J. Avigad, *The mechanization of mathematics*, Notices of the AMS **65** (2018), 681–690.

[6] K. Buzzard, *The future of mathematics?*, ICM 2022.

[7] The Mathlib Community, *Mathlib4*, `https://github.com/leanprover-community/mathlib4`.